

Welcome to the Production Anomaly Detection Project!

This documentation provides a complete guide to the real-time anomaly detection system. This page focuses specifically on the main Flask web application that drives the user interface and simulation.

API Reference:

- [project](#)
 - [project package](#)

Indices and tables

- [Index](#)
- [Module Index](#)
- [Search Page](#)

Market Event Predictor

Go

Navigation

API Reference:

- [project](#)

Related Topics

- [Documentation overview](#)
 - Next: [project](#)

project.app package

Submodules

project.app.app module

project.app.create_artifacts module

project.app.create_artifacts.create_and_save_artifacts()[\[source\]](#)

Loads training data, creates preprocessing artifacts compatible with the ProductionAnomalyPredictor, and saves them to a pickle file.

Module contents

Market Event Predictor

Go

Navigation

API Reference:

- [project](#)
 - [project package](#)

Related Topics

- [Documentation overview](#)
 - [project](#)
 - [project package](#)
 - Previous: [project package](#)
 - Next: [project.config package](#)

project.config package

Submodules

project.config.model_config module

class project.config.model_config.LSTMConfig(*train_path: str, test_path: str, model_save_path: str, non_feature_cols: List[str] = None*)[\[source\]](#)

Bases: [object](#)

Configuration class for LSTM market event detection model.

batch_norm: bool = True[\[\]](#)

batch_size: int = 128[\[\]](#)

dropout_rate: float = 0.2[\[\]](#)

epochs: int = 100[\[\]](#)

learning_rate: float = 0.0003[\[\]](#)

lr_reduction_factor: float = 0.5[\[\]](#)

model_save_path: str = None[\[\]](#)

non_feature_cols: List[str] = None[\[\]](#)

patience_early_stopping: int = 8[\[\]](#)

patience_lr_reduction: int = 8[\[\]](#)

sequence_length: int = 30[\[\]](#)

target_column: str = 'event_in_30min'[\[\]](#)

test_path: str = None[\[\]](#)

train_path: str = None[\[\]](#)

use_class_weights: bool = True[\[\]](#)

val_ratio: float = 0.5[\[\]](#)

class project.config.model_config.Stage1Config(*sequence_length: int = 30, val_ratio: float = 0.5, target_column: str = 'event_in_30min', non_feature_cols: List[str] = None, dropout_rate: float = 0.3, batch_norm: bool = True, epochs: int = 50, batch_size: int = 128, learning_rate: float = 0.0003, use_class_weights: bool = True, train_path: str = None, test_path: str = None, model_save_path: str = None, patience_early_stopping: int = 8, patience_lr_reduction: int = 8, lr_reduction_factor: float = 0.5*)[\[source\]](#)

Bases: [LSTMConfig](#)

Configuration for Stage 1 binary classifier.

dropout_rate: float = 0.3[\[\]](#)

epochs: int = 50[\[\]](#)

class project.config.model_config.Stage2Config(*sequence_length: int = 30, val_ratio: float = 0.5, target_column: str = 'event_in_30min', non_feature_cols: List[str] = None, dropout_rate: float = 0.4, batch_norm: bool = True, epochs: int = 80, batch_size: int = 128, learning_rate: float = 0.0003, use_class_weights: bool = True, train_path: str = None, test_path: str = None, model_save_path: str = None, patience_early_stopping: int = 8, patience_lr_reduction: int = 8, lr_reduction_factor: float = 0.5, use_smote: bool = True*)[\[source\]](#)

Bases: [LSTMConfig](#)

Configuration for Stage 2 anomaly classifier.

dropout_rate: float = 0.4[\[\]](#)

epochs: int = 80[\[\]](#)

use_smote: bool = True[\[\]](#)

Module contents

project.data.preprocessing package

Submodules

project.data.preprocessing.data_processor module

`class project.data.preprocessing.data_processor.LSTMDataProcessor(config)` [\[source\]](#)

Bases: `object`

Handles data loading and preprocessing for LSTM model.

This class provides functionality for loading time series data, validating datasets, creating sequential data for LSTM training, and performing various preprocessing tasks including scaling and encoding.

`config` [\[source\]](#)

Configuration object containing model parameters and file paths.

`scaler` [\[source\]](#)

MinMaxScaler instance for feature normalization.

`label_encoder` [\[source\]](#)

LabelEncoder instance for target variable encoding.

`load_and_validate_data()` → `Tuple[DataFrame, DataFrame]` [\[source\]](#)

Load training and test data with validation.

Loads CSV files specified in the configuration, converts date columns to datetime, sets proper indexing, sorts by date, and validates compatibility between datasets.

Returns:

A tuple containing:

- `df_train`: Training dataset with datetime index
- `df_test`: Test dataset with datetime index

Return type:

`Tuple[pd.DataFrame, pd.DataFrame]`

Raises:

Exception – If there's an error loading files or processing data.

Example

```
>>> processor = LSTMDataProcessor(config)
>>> train_df, test_df = processor.load_and_validate_data()
```

`prepare_sequences(df: DataFrame, fit_scalers: bool = True, label_encoder: LabelEncoder = None)` → `Tuple[ndarray, ndarray]` [\[source\]](#)

Prepare sequential data for LSTM training.

Transforms time series data into sequential format suitable for LSTM training, including feature scaling, label encoding, and sequence creation.

Parameters:

- **df** – Input DataFrame with time series data.
- **fit_scalers** – Whether to fit scalers on this data (True for training, False for validation/test data).
- **label_encoder** – Optional custom label encoder to use instead of the instance's label_encoder.

Returns:

A tuple containing:

- `X_seq`: 3D array of shape (samples, sequence_length, features)

containing input sequences

- `y_categorical`: 2D array of shape (samples, num_classes) containing one-hot encoded target labels

Return type:

Tuple[np.ndarray, np.ndarray]

Raises:

Exception – If there's an error during sequence preparation or scaling.

Example

```
>>> X_train, y_train = processor.prepare_sequences(train_df, fit_scalers=True)
>>> X_test, y_test = processor.prepare_sequences(test_df, fit_scalers=False)
```

`split_temporal_validation(df: DataFrame) → Tuple[DataFrame, DataFrame]`[\[source\]](#)[¶](#)

Split test data temporally for validation.

Performs a temporal split of the dataset, preserving time order by taking the first portion for training/validation and the later portion for testing.

Parameters:

df – Input DataFrame to split temporally.

Returns:

A tuple containing:

- First portion of the data (earlier timestamps)
- Second portion of the data (later timestamps)

Return type:

Tuple[pd.DataFrame, pd.DataFrame]

Note

The split ratio is determined by `self.config.val_ratio`. This maintains temporal order which is crucial for time series validation.

`class project.data.preprocessing.data_processor.TwoStageLSTMDataProcessor(config)`[\[source\]](#)[¶](#)

Bases: [LSTMDataProcessor](#)

Extended data processor for two-stage approach.

This class extends `LSTMDataProcessor` to support a two-stage classification approach: Stage 1: Binary classification (normal vs anomaly) Stage 2: Multi-class classification of anomaly types

This approach can improve performance when dealing with imbalanced datasets where normal samples significantly outnumber anomaly samples.

`anomaly_label_encoder`[¶](#)

`LabelEncoder` specifically for anomaly types in Stage 2.

`prepare_anomaly_only_sequences(df: DataFrame)`[\[source\]](#)[¶](#)

Prepare sequences with only anomaly samples for Stage 2.

Filters the dataset to include only anomaly samples (excluding “normal” class) and creates a new label encoder specifically for anomaly types.

Parameters:

df – Input DataFrame with time series data.

Returns:

A tuple containing:

- `X_seq`: 3D array of input sequences from anomaly samples only

- `y_seq`: 2D one-hot encoded labels for anomaly types only

Return type:

`Tuple[np.ndarray, np.ndarray]`

Raises:

ValueError – If no anomaly samples are found in the dataset.

Note

This method creates and stores a new `anomaly_label_encoder` attribute that maps only the anomaly class types (excluding “normal”).

Example

```
>>> X_anomaly, y_anomaly = processor.prepare_anomaly_only_sequences(train_df)
>>> # Only samples with non-"normal" labels are included
```

`prepare_binary_sequences(df: DataFrame, fit_scalers: bool = True)`[\[source\]](#)[¶](#)

Prepare binary sequences for Stage 1 (Normal vs Anomaly).

Converts multi-class labels into binary labels for the first stage of classification, where all non-normal classes are grouped as anomalies.

Parameters:

- **df** – Input DataFrame with time series data.
- **fit_scalers** – Whether to fit scalers on this data.

Returns:

A tuple containing:

- `X_seq`: 3D array of input sequences
- `y_binary_cat`: 2D one-hot encoded binary labels (normal=0, anomaly=1)

Return type:

`Tuple[np.ndarray, np.ndarray]`

Note

This method assumes that “normal” is one of the classes in the label encoder. All other classes are treated as anomalies.

Example

```
>>> X_binary, y_binary = processor.prepare_binary_sequences(train_df)
>>> # y_binary will have shape (samples, 2) for binary classification
```

project.data.preprocessing.generate_dataset module[¶](#)

`project.data.preprocessing.generate_dataset.add_target_variable(df: DataFrame, priority: Dict[str, int], horizon_minutes: int) → DataFrame`[\[source\]](#)[¶](#)

Add the target variable for prediction based on future events within a time horizon.

Creates a new target column that contains the most important event occurring within a specified time horizon from each timestamp. The importance is determined by the priority dictionary, with higher values indicating higher priority events.

Parameters:

- **df** – Input DataFrame with time series data containing an ‘event’ column.
- **priority** – Dictionary mapping event names to their priority levels (higher = more important). Example: `{"normal": 0, "dip": 1, "rally": 2, "crash": 3}`
- **horizon_minutes** – Time horizon in minutes to look ahead for future events.

Returns:

DataFrame with the original ‘event’ column removed and a new

'event_in_30min' column containing the target variable.

Return type:

pd.DataFrame

Note

The target variable represents the most important event that will occur within the specified time horizon from each timestamp. If no events are found within the horizon, the lowest priority event is assigned.

Example

```
>>> priority = {"normal": 0, "crash": 3}
>>> df_with_target = add_target_variable(df, priority, 30)
>>> # New column 'event_in_30min' contains future events within 30 minutes
```

```
project.data.preprocessing.generate_dataset.generate_dataset(input_file: str, output_file_train: str, output_file_test: str,
priority: Dict[str, int], horizon_minutes: int = 30, cutoff_date: str = '2024-12-31', test_size: float = 0.2) → None \[source\]
```

Generate a dataset for LSTM training with temporal target variable creation.

This is the main function that orchestrates the entire dataset generation process for time series prediction. It loads raw data, creates predictive target variables based on future events within a time horizon, splits the data temporally, and saves the processed datasets.

The function is designed for financial time series prediction where the goal is to predict market events (normal, dip, rally, crash) based on historical features.

Parameters:

- **input_file** - Path to input CSV file containing raw time series data with 'event' column.
- **output_file_train** - Path where the training dataset will be saved.
- **output_file_test** - Path where the testing dataset will be saved.
- **priority** - Dictionary mapping event names to priority levels for determining the most important event within the time horizon. Example: {"normal": 0, "dip": 1, "rally": 2, "crash": 3}
- **horizon_minutes** - Time horizon in minutes for looking ahead to create target variables. Default is 30 minutes.
- **cutoff_date** - ISO format date string for filtering data up to this date. Default is "2024-12-31".
- **test_size** - Proportion of data to use for testing (0.0 to 1.0). Default is 0.2 (20%).

Raises:

Exception - If any step in the dataset generation process fails.

Note

This function implements a complete pipeline for creating supervised learning datasets from time series data. The target variable represents the most important event that will occur within the specified time horizon.

Example

```
>>> priority = {"normal": 0, "dip": 1, "rally": 2, "crash": 3}
>>> generate_dataset(
...     "raw_data.csv",
...     "train_data.csv",
...     "test_data.csv",
...     priority,
...     horizon_minutes=30,
...     test_size=0.2
... )
# Creates training and testing datasets for 30-minute ahead prediction
```

```
project.data.preprocessing.generate_dataset.load_and_prepare_data(input_file: str, cutoff_date: str) →
DataFrame \[source\]
```

Load CSV data and prepare it with proper datetime indexing.

Loads a CSV file containing time series data, converts the Date column to datetime format, sets it as the index, sorts the data chronologically, and applies a cutoff date filter.

Parameters:

- **input_file** - Path to the input CSV file containing time series data.

- **cutoff_date** – ISO format date string (e.g., “2024-12-31”) to filter data up to this date.

Returns:

A DataFrame with datetime index, sorted chronologically and filtered by the cutoff date.

Return type:

pd.DataFrame

Raises:

- **ValueError** – If the DataFrame index is not properly sorted after processing.
- **FileNotFoundError** – If the input file doesn’t exist.

Example

```
>>> df = load_and_prepare_data("data.csv", "2024-12-31")
>>> print(df.index.is_monotonic_increasing) # True
```

project.data.preprocessing.generate_dataset.log_distribution_summary(df_train: DataFrame, df_test: DataFrame, target_column: str) → None[\[source\]](#)

Log distribution summary of target variable in train and test sets.

Analyzes and logs the distribution of the target variable across both training and testing datasets. This helps identify potential data imbalance issues and ensures that both datasets have representative distributions.

Parameters:

- **df_train** – Training dataset containing the target variable.
- **df_test** – Testing dataset containing the target variable.
- **target_column** – Name of the target column to analyze.

Note

The distributions are logged as proportions (normalized) to make it easy to compare class balance between training and testing sets.

Example

```
>>> log_distribution_summary(train_df, test_df, "event_in_30min")
# Logs proportions of each event type in both train and test sets
```

project.data.preprocessing.generate_dataset.save_datasets(df_train: DataFrame, df_test: DataFrame, output_file_train: str, output_file_test: str) → None[\[source\]](#)

Save training and testing datasets to CSV files.

Exports the processed training and testing datasets to separate CSV files, preserving the datetime index for future loading and processing.

Parameters:

- **df_train** – Training dataset to save.
- **df_test** – Testing dataset to save.
- **output_file_train** – File path for saving the training dataset.
- **output_file_test** – File path for saving the testing dataset.

Note

The index (datetime) is preserved in the saved files to maintain temporal information when the datasets are loaded later.

Example

```
>>> save_datasets(train_df, test_df, "train.csv", "test.csv")
# Creates train.csv and test.csv with preserved datetime indices
```

project.data.preprocessing.generate_dataset.split_dataset(df: DataFrame, test_size: float) → Tuple[DataFrame, DataFrame][\[source\]](#)

Split dataset into training and testing sets using temporal splitting.

Performs a temporal split of the dataset, preserving chronological order by taking the first portion for training and the later portion for testing. This is crucial for time series data to avoid data leakage.

Parameters:

- **df** – Input DataFrame with time series data (must be chronologically sorted).
- **test_size** – Proportion of data to use for testing (between 0.0 and 1.0).

Returns:

A tuple containing:

- **df_train**: Training dataset (earlier timestamps)
- **df_test**: Testing dataset (later timestamps)

Return type:

Tuple[pd.DataFrame, pd.DataFrame]

Note

The split maintains temporal order, which is essential for time series modeling to ensure that the model is not trained on future data when predicting the past.

Example

```
>>> train_df, test_df = split_dataset(df, test_size=0.2)
>>> # 80% of data (earliest) for training, 20% (latest) for testing
```

project.data.preprocessing.initial_data_engineering module¶

project.data.preprocessing.initial_data_engineering.calculate_atr(*df_slice: DataFrame, window: int = 14*) → Series[\[source\]](#)¶

Calculates the Average True Range (ATR).

ATR is a technical analysis indicator that measures market volatility by decomposing the entire range of an asset price for that period.

Parameters:

- **df_slice** – A pandas DataFrame containing 'High', 'Low', and 'Close' columns.
- **window** – The lookback period for the ATR calculation. Default is 14.

Returns:

A pandas Series containing the calculated ATR values.

project.data.preprocessing.initial_data_engineering.calculate_rsi(*series: Series, window: int = 14*) → Series[\[source\]](#)¶

Calculates the Relative Strength Index (RSI) for a given series.

RSI is a momentum indicator that measures the magnitude of recent price changes to evaluate overbought or oversold conditions.

Parameters:

- **series** – A pandas Series of prices (e.g., closing prices).
- **window** – The lookback period for the RSI calculation. Default is 14.

Returns:

A pandas Series containing the calculated RSI values.

project.data.preprocessing.initial_data_engineering.detect_events_updated(*df_orig: DataFrame*) → DataFrame[\[source\]](#)¶

Labels each row as 'crash', 'dip', 'rally', or 'normal' based on future price changes.

This function looks ahead in the data to identify significant price movements within defined time windows and magnitude thresholds.

Parameters:

df_orig - The input DataFrame with a 'Close' price column.

Returns:

The DataFrame with an added 'event' column labeling each time point.

`project.data.preprocessing.initial_data_engineering.load_and_prepare_minute_data(file_path: str) → DataFrame`[\[source\]](#)[¶](#)

Loads minute-level data, filters for trading hours, and handles missing data.

This function reads a CSV, filters out non-trading hours, creates a complete index for all trading minutes, and interpolates missing values.

Parameters:

file_path - The path to the minute-level data CSV file.

Returns:

A cleaned and prepared pandas DataFrame with minute-level data.

`project.data.preprocessing.initial_data_engineering.preprocess_data()`[\[source\]](#)[¶](#)

Main function to execute the full data processing and feature engineering pipeline.

Module contents¶

Market Event Predictor

Navigation

API Reference:

- [project](#)
 - [project package](#)

Related Topics

- [Documentation overview](#)
 - [project](#)
 - [project package](#)
 - [project.data package](#)
 - Previous: [project.data package](#)
 - Next: [project.models package](#)

project.models package

Submodules

project.models.callbacks module

`class project.models.callbacks.ClassificationReportCallback(X_val, y_val, label_encoder)`[\[source\]](#)

Bases: `Callback`

Custom callback to print detailed classification reports during training.

This callback generates and prints comprehensive classification reports at regular intervals during training, providing insights into model performance across different classes.

The callback converts model predictions and true labels from one-hot encoded format to class indices and generates detailed metrics including precision, recall, and F1-score for each class.

`X_val`[\[link\]](#)

Validation input data for generating predictions.

`y_val`[\[link\]](#)

Validation target data (one-hot encoded) for comparison.

`label_encoder`[\[link\]](#)

Label encoder with class names for meaningful report display.

`on_epoch_end(epoch, logs=None)`[\[source\]](#)[\[link\]](#)

Generate and print classification report at the end of specified epochs.

Parameters:

- **epoch** - Current epoch number (0-indexed).
- **logs** - Dictionary containing training metrics for the current epoch.

Note

The classification report includes precision, recall, F1-score, and support for each class, providing a comprehensive view of model performance across all classes in the dataset.

Example Output:

```
=== CLASSIFICATION REPORT (Epoch 10) ===  
  
precision recall f1-score support  
  
normal 0.9500 0.9200 0.9347 100 dip 0.7800 0.8100 0.7949 37 rally 0.8200 0.7900 0.8047 38 crash  
0.9100 0.9500 0.9295 20
```

`class project.models.callbacks.OverfittingMonitorCallback(patience=3, threshold=0.2)`[\[source\]](#)[\[link\]](#)

Bases: `Callback`

Monitor for overfitting and automatically stop training when detected.

This callback monitors the difference between training and validation loss to detect overfitting patterns. When the validation loss becomes significantly higher than the training loss for a sustained period, it indicates that the model is overfitting to the training data and losing its ability to generalize.

The callback implements a patience mechanism where overfitting must be detected for consecutive epochs before stopping training, preventing false positives from temporary fluctuations in the loss metrics.

`patience`[\[link\]](#)

Number of consecutive epochs with overfitting before stopping training.

`threshold`[\[link\]](#)

Minimum difference between validation and training loss to consider overfitting.

`wait`[\[link\]](#)

Current count of consecutive epochs with detected overfitting.

`on_epoch_end(epoch, logs=None)`[\[source\]](#)[¶](#)

Monitor training and validation loss to detect overfitting patterns.

Parameters:

- **epoch** – Current epoch number (0-indexed).
- **logs** – Dictionary containing training metrics including 'loss' and 'val_loss'.

Note

The callback waits until epoch 5 before starting monitoring to allow the model to stabilize its learning patterns. Early epochs often show high variance in loss values that could trigger false overfitting detection.

Warning

When overfitting is detected and training is stopped, the callback logs both the detection event and the current loss values for debugging purposes.

Example

If validation loss is 0.8 and training loss is 0.5 with threshold=0.2: - Difference: $0.8 - 0.5 = 0.3 > 0.2$ (threshold) - Overfitting detected, increment wait counter - If wait $\geq$ patience, stop training

project.models.evaluate module[¶](#)

`project.models.evaluate.evaluate_model(trainer, df_test, labels_mapping: Dict = None) → Tuple[float, float, float]`[\[source\]](#)[¶](#)

Comprehensive evaluation function for two-stage LSTM anomaly detection models.

This function evaluates a trained two-stage LSTM model on test data, computing key performance metrics that are critical for anomaly detection systems in financial markets. It handles both numeric and string label mappings and provides a simple pass/fail verdict based on predefined thresholds.

The evaluation focuses on three critical metrics: 1. Overall accuracy - General model performance across all classes 2. Crash detection rate (recall) - Ability to detect critical events 3. False alarm rate - Frequency of incorrectly flagging normal conditions as anomalies

Parameters:

- **trainer** – Trained TwoStageLSTMTrainer instance containing both stage models and the data processor with fitted scalers and encoders.
- **df_test** – Test dataset as DataFrame with datetime index and all required features.
- **labels_mapping** – Optional mapping for label conversion. Can be: - Dict mapping indices to string labels - Array of string labels - None to use default numeric labels

Returns:

A tuple containing:

- **overall_accuracy**: Overall classification accuracy (0.0 to 1.0)
- **crash_recall**: Recall for detecting crash events (0.0 to 1.0)
- **false_alarm_rate**: Rate of false positives for normal class (0.0 to 1.0)

Return type:

Tuple[float, float, float]

Note

The function includes robust error handling and returns sentinel values (0.0, 0.0, 1.0) in case of evaluation failure, indicating worst-case performance metrics.

Example

```
>>> accuracy, crash_recall, false_alarms = evaluate_model(
...     trainer, test_df, labels_mapping=['normal', 'dip', 'rally', 'crash']
... )
>>> print(f"Accuracy: {accuracy:.1%}, Crash Recall: {crash_recall:.1%}")
```

Success Criteria:

The system is considered ready for deployment if: - Overall accuracy > 80% - Crash detection rate > 70% - False alarm rate < 10%

`project.models.evaluate.make_two_stage_predictions(trainer, X_test, labels_mapping: Dict = None) → Tuple[ndarray, ndarray]`[\[source\]](#)[¶]

Generate predictions using the two-stage LSTM classification approach.

This function implements the core two-stage prediction pipeline:

Stage 1: Binary Anomaly Detection - Uses the first LSTM model to classify samples as normal vs anomaly - Outputs probability scores for anomaly detection confidence

Stage 2: Anomaly Type Classification - For samples classified as anomalies in Stage 1, uses the second LSTM model - Classifies the specific type of anomaly (dip, rally, crash, etc.) - Only processes samples that were flagged as anomalies

This approach improves performance on imbalanced datasets by first separating normal from abnormal conditions, then focusing detailed classification on anomalies only.

Parameters:

- **trainer** - Trained TwoStageLSTMTrainer instance containing: - stage1_model: Binary classification model (normal vs anomaly) - stage2_model: Multi-class model for anomaly types
- **X_test** - Test input sequences of shape (samples, sequence_length, features).
- **labels_mapping** - Optional mapping for converting internal predictions to meaningful labels. Can be: - Dict: {index: label_name} - Array: [label_name_0, label_name_1, ...] - None: Use numeric indices

Returns:

A tuple containing:

- final_predictions: Array of final class predictions for each sample
- anomaly_confidence: Array of anomaly probability scores from Stage 1

Return type:

`Tuple[np.ndarray, np.ndarray]`

Raises:

Exception - If there's an error during prediction process.

Note

The function handles label mapping intelligently: - String labels are converted to appropriate integer representations - Normal class is always mapped to index 0 - Anomaly types are mapped to indices 1, 2, 3, etc.

Example

```
>>> predictions, confidence = make_two_stage_predictions(
...     trainer, X_test, labels_mapping=['normal', 'dip', 'rally', 'crash']
... )
>>> print(f"Detected {np.sum(predictions != 0)} anomalies")
>>> print(f"Average confidence: {np.mean(confidence):.2f}")
```

Processing Flow:

1. Stage 1 processes all samples → binary predictions (normal/anomaly)
2. Extract samples predicted as anomalies
3. Stage 2 processes only anomaly samples → specific anomaly types
4. Combine results: normal=0, anomaly types=1,2,3...
5. Apply label mapping if provided

project.models.lstm module[¶](#)

`project.models.lstm.train_two_stage_lstm(train_path: str, test_path: str, stage1_model_path: str, stage2_model_path: str) → Tuple[Tuple[DataFrame, DataFrame], Tuple[Any, Any], Tuple[float, float, float], TwoStageLSTMTrainer]`[\[source\]](#)[¶]

Train a complete two-stage LSTM system for financial anomaly detection.

This function orchestrates the entire training pipeline for a two-stage LSTM anomaly detection system designed

for financial time series data. The system uses a hierarchical approach where the first stage performs binary classification (normal vs anomaly) and the second stage classifies specific anomaly types.

The two-stage approach is particularly effective for imbalanced datasets common in financial markets, where normal conditions significantly outnumber anomalous events. By separating the detection and classification tasks, each model can be optimized for its specific purpose.

Training Pipeline: 1. Initialize configurations for both stages 2. Load and validate training and test datasets 3. Split test data into validation and final test sets (temporal split) 4. Train Stage 1: Binary anomaly detection model 5. Train Stage 2: Multi-class anomaly classification model 6. Evaluate the complete system on final test set 7. Save production-ready artifacts

Parameters:

- **train_path** – Path to the training dataset CSV file. Must contain datetime-indexed time series data with features and target labels.
- **test_path** – Path to the test dataset CSV file. Should have the same structure as the training data for proper validation.
- **stage1_model_path** – File path where the binary classification model will be saved. Should use .keras extension for TensorFlow/Keras models.
- **stage2_model_path** – File path where the anomaly classification model will be saved. Should use .keras extension for TensorFlow/Keras models.

Returns:

- (df_train, df_test): Tuple of loaded training and test DataFrames
- (history1, history2): Tuple of training histories for both stages
- evaluation: Tuple of (accuracy, crash_recall, false_alarm_rate) metrics
- trainer: Trained TwoStageLSTMTrainer instance with fitted models

Return type:

Tuple containing four elements

Raises:

- **FileNotFoundError** – If training or test data files cannot be found.
- **ValueError** – If data validation fails or datasets are incompatible.
- **Exception** – If training fails for either stage or evaluation encounters errors.

Note

The function automatically saves production artifacts including: - Trained models (stage1_lstm.keras, stage2_lstm.keras) - Preprocessing components (scalers, encoders) - Configuration objects - Model metadata

Example

```
>>> dfs, histories, metrics, trainer = train_two_stage_lstm(
...     "data/train.csv",
...     "data/test.csv",
...     "models/stage1.keras",
...     "models/stage2.keras"
... )
>>> accuracy, crash_recall, false_alarms = metrics
>>> print(f"System accuracy: {accuracy:.1%}")
```

project.models.predict module¶

`class project.models.predict.ProductionAnomalyPredictor(models_path: str = 'models/saved_models/')` [\[source\]¶](#)

Bases: object

Production-ready anomaly predictor for real-time financial market surveillance. This version uses a predefined feature list for robustness and an efficient NumPy buffer for performance.

`predict_single_point(new_data_point: dict)` [\[source\]¶](#)

Predict using the efficient NumPy buffer and a fixed feature list.

`reset_buffer()` [\[source\]¶](#)

Reset the rolling buffer.

project.models.two_stage_classifier module

`class project.models.two_stage_classifier.TwoStageLSTMTrainer(stage1_config, stage2_config)`[\[source\]](#)

Bases: `object`

Comprehensive trainer for two-stage LSTM anomaly detection system.

This class manages the complete training pipeline for a hierarchical anomaly detection system designed for financial time series. The two-stage approach addresses the challenges of imbalanced datasets common in financial markets:

Stage 1: Binary Classification (Normal vs Anomaly) - Optimized for high sensitivity to detect any anomalous behavior - Uses balanced training with class weights - Focuses on not missing any potential anomalies

Stage 2: Anomaly Type Classification (Crash/Dip/Rally/etc.) - Specialized for distinguishing between different anomaly types - Trained only on pre-identified anomalous samples - Can use techniques like SMOTE for better class balance

The trainer handles all aspects of the training process including: - Data preprocessing and sequence preparation - Model architecture selection and compilation - Advanced callback management for robust training - Class weight computation for imbalanced datasets - Production artifact saving for deployment

`stage1_config`[\[link\]](#)

Configuration object for binary classification stage.

`stage2_config`[\[link\]](#)

Configuration object for anomaly classification stage.

`stage1_model`[\[link\]](#)

Trained binary classification model.

`stage2_model`[\[link\]](#)

Trained anomaly type classification model.

`data_processor`[\[link\]](#)

Data preprocessing and sequence preparation component.

`anomaly_label_encoder`[\[link\]](#)

Encoder for anomaly type labels.

`stage1_history`[\[link\]](#)

Training history from Stage 1.

`stage2_history`[\[link\]](#)

Training history from Stage 2.

`save_production_artifacts(base_path: str = 'models/saved_models/') → str`[\[source\]](#)[\[link\]](#)

Save all components needed for production inference deployment.

Creates a complete production deployment package containing all trained models and preprocessing components required for inference. This method ensures that the production predictor can operate independently without access to the training environment or training data.

Parameters:

base_path - Directory path where all production artifacts will be saved. Directory will be created if it doesn't exist.

Returns:

Path to the saved preprocessing artifacts file.

Return type:

str

Raises:

Exception - If saving fails due to file system issues or missing components.

Saved Components:

- stage1_lstm.keras: Binary anomaly detection model
- stage2_lstm.keras: Anomaly type classification model
- preprocessing_artifacts.pkl: All preprocessing components including: * Fitted data scalers * Label encoders * Configuration objects * Feature column definitions * Sequence length parameters

Note

This method creates a complete, self-contained deployment package. The saved artifacts can be loaded by ProductionAnomalyPredictor for real-time inference without any dependencies on training code.

Example

```
>>> artifacts_path = trainer.save_production_artifacts("deployment/")
>>> print("Ready for production deployment!")
```

`train_stage1(df_train, df_val)`[\[source\]](#)[¶](#)

Train binary anomaly detection model with callbacks.

The training process includes: 1. Data preparation with binary sequence generation 2. Model architecture building and compilation 3. Class weight computation for balanced training 4. Comprehensive callback setup for robust training 5. Model training with monitoring and optimization

Parameters:

- **df_train** - Training dataset DataFrame with datetime index and features.
- **df_val** - Validation dataset DataFrame for monitoring training progress.

Returns:

Training history object containing loss and metric values throughout training, useful for plotting learning curves and diagnosing training.

Raises:

Exception - If training fails due to data issues, model problems, or insufficient resources.

Note

The method automatically handles class imbalance through computed weights. The trained model is automatically saved to the configured path.

Example

```
>>> history = trainer.train_stage1(train_df, val_df)
>>> print(f"Final validation accuracy: {history.history['val_accuracy'][-1]:.3f}")
```

`train_stage2(df_train, df_val)`[\[source\]](#)[¶](#)

Train anomaly type classification model with callbacks.

The training process includes: 1. Anomaly-only data preparation and sequence generation 2. Optional SMOTE oversampling for better class balance 3. Complex model architecture building and compilation 4. Class weight computation and callback setup 5. Specialized training for anomaly type discrimination

Parameters:

- **df_train** - Training dataset DataFrame with datetime index and features.
- **df_val** - Validation dataset DataFrame for monitoring training progress.

Returns:

Training history object containing loss and metric values throughout training, useful for analyzing training dynamics and model performance.

Raises:

Exception - If training fails due to insufficient anomaly samples, SMOTE issues, or model training problems.

Note

This stage only trains on anomalous samples, which may result in smaller training sets. SMOTE oversampling can be enabled via configuration to improve class balance among different anomaly types.

Example

```
>>> history = trainer.train_stage2(train_df, val_df)
>>> anomaly_types = trainer.anomaly_label_encoder.classes_
>>> print(f"Trained to classify: {list(anomaly_types)}")
```

`project.models.two_stage_classifier.build_stage1_lstm_model(input_shape: Tuple[int, int]) → Sequential`[\[source\]](#)[¶](#)

Build a simplified LSTM model for binary anomaly detection.

This function creates a convolutional-LSTM hybrid architecture optimized for binary classification of time series data into normal vs anomalous patterns. The model uses a combination of: - 1D convolution for local pattern detection - Bidirectional LSTM for temporal dependency modeling - Regularization techniques to prevent overfitting

Parameters:

input_shape - Tuple of (sequence_length, n_features) defining the input dimensions for the LSTM model.

Returns:

Compiled Keras model ready for training with binary
classification output (normal vs anomaly).

Return type:

Sequential

Architecture Components:

- Conv1D layer: Captures local temporal patterns
- MaxPooling1D: Reduces sequence length for efficiency
- Bidirectional LSTM: Models temporal dependencies in both directions
- Dense layers: Final classification with regularization
- Dropout/BatchNormalization: Prevents overfitting

Example

```
>>> model = build_stage1_lstm_model((30, 10))
>>> print(f"Model has {model.count_params()} parameters")
```

`project.models.two_stage_classifier.build_stage2_lstm_model(input_shape: Tuple[int, int], num_anomaly_classes: int) → Sequential`[\[source\]](#)[¶](#)

Build a specialized LSTM model for anomaly type classification.

This function creates a more complex LSTM architecture designed specifically for multi-class classification of different anomaly types. The model is deeper and more sophisticated than Stage 1, as it needs to distinguish between subtle differences in anomaly patterns (e.g., crash vs rally vs dip).

The architecture incorporates: - Multiple convolutional layers for complex pattern extraction - Stacked bidirectional LSTMs for deep temporal modeling - Extensive regularization for stable training on imbalanced data - Multiple dense layers for refined classification

Parameters:

- **input_shape** - Tuple of (sequence_length, n_features) defining the input dimensions for the LSTM model.
- **num_anomaly_classes** - Number of different anomaly types to classify (e.g., 3 for crash/dip/rally).

Returns:

Compiled Keras model ready for training with multi-class
classification output for anomaly types.

Return type:

Sequential

Architecture Components:

- Multiple Conv1D layers: Extract complex local patterns
- Stacked bidirectional LSTMs: Deep temporal sequence modeling
- Multiple dense layers: Hierarchical feature refinement
- Extensive dropout/batch normalization: Robust training

Example

```
>>> model = build_stage2_lstm_model((30, 10), 3)
>>> print(f"Model classifies {num_anomaly_classes} anomaly types")
```

Module contents¶

Market Event Predictor

Go

Navigation

API Reference:

- [project](#)
 - [project package](#)

Related Topics

- [Documentation overview](#)
 - [project](#)
 - [project package](#)
 - Previous: [project.data.preprocessing package](#)
 - Next: [project.utils package](#)

project.utils package

Submodules

project.utils.processing_helpers module

project.utils.processing_helpers.get_event_in_horizon(df: DataFrame, y_col_name: str, timestamp: Timestamp, horizon_minutes: int, priority: Dict[str, int]) → str

Get the most important event label within a specified time horizon.

Parameters:

- **df** - DataFrame containing time series data with datetime index and event labels. Must be sorted by timestamp for proper time-based slicing.
- **y_col_name** - Name of the column containing event labels (e.g., 'event_type', 'market_condition'). This column should contain categorical labels representing different types of market events.
- **timestamp** - Starting timestamp for the look-ahead window. Must be a pandas Timestamp object that exists in the DataFrame's index or can be used for time-based slicing.
- **horizon_minutes** - Length of the forward-looking time window in minutes. Typical values might be 30, 60, or 120 minutes depending on the prediction horizon required for the application.
- **priority** - Dictionary mapping event label names to integer priority values. Higher values indicate more important events. For example: {'normal': 1, 'dip': 2, 'rally': 3, 'crash': 4} This ensures crashes are prioritized over dips when both occur.

Returns:

Event label with the highest priority within the specified time horizon.

If no events are found in the horizon, returns the event with the lowest priority value (typically 'normal' conditions).

Return type:

str

Raises:

- **KeyError** - If y_col_name doesn't exist in the DataFrame columns.
- **ValueError** - If timestamp is not valid for DataFrame slicing.
- **TypeError** - If priority dictionary values are not numeric.

Example

```
>>> priority = {'normal': 1, 'dip': 2, 'rally': 3, 'crash': 4}
>>> event = get_event_in_horizon(
...     df=market_data,
...     y_col_name='event_type',
...     timestamp=pd.Timestamp('2024-01-15 10:00:00'),
...     horizon_minutes=30,
...     priority=priority
... )
>>> print(f"Most critical event in next 30 minutes: {event}")
```

Technical Details:

The time window is defined as (timestamp + 1 second, timestamp + horizon_minutes]. This ensures the current timestamp is excluded from the search while including all events up to and including the end of the horizon period.

Module contents

Market Event Predictor

Go